Ludwig-Maximilians-Universität München

Institut für Informatik

Dr. Thomas Lemberger

2. Probeklausur Softwaretechnik Wintersemester 2025/26

München, 28. Januar 2026

Hinweise:

- Notieren Sie bitte Ihren Namen und Ihre Matrikelnummer auf jedem Aufgabenblatt.
- Notieren Sie Ihre Lösung jeweils unter der Aufgabe auf dem **jeweiligen Aufgabenblatt**.
  Nutzen Sie gegebenenfalls auch die **Rückseite**.
- Geben Sie auch alle **unbearbeiteten Aufgabenblätter** ab.
- Verwenden Sie jedes Blatt inklusive Rückseite nur für **die jeweilige Aufgabe**.

Zusatzblätter:

- Ein **Zusatzblatt** finden Sie am Ende der Klausur, weitere Blätter erhalten Sie bei der Klausuraufsicht.
- Schreiben Sie auf jedes Zusatzblatt **Ihren Namen**, **Ihre Matrikelnummer** und **die Nummer der Aufgabe**.
- Die Verwendung eigener Blätter ist verboten.
- Verwenden Sie jedes Zusatzblatt nur für **eine Aufgabe**.

Sonstiges:

- Schalten Sie alle Ihre elektronischen Geräte aus.
- Legen Sie Ihren Studierendenausweis und einen amtlichen Ausweis auf den Tisch.
- Schreiben Sie **leserlich** und mit einem blauen oder schwarzen Stift.
- Geben Sie **nur eine Lösung je Aufgabe** ab
  Falls Sie mehr als eine Lösung einreichen, wird die **schlechteste** bewertet.
- Sie haben 45 Minuten Zeit.
- Es sind 30 Punkte erreichbar.
- Während der Klausur dürfen Sie **keinerlei Hilfsmittel** verwenden.

Viel Erfolg!

Wird vom Lehrstuhl ausgefüllt

Die Klausur besteht aus zwei Teilen: einem Multiple-Choice-Teil und einem Teil mit freien Aufgaben.
Der freie Teil hat 4 Aufgaben.

Aufgabe	MC	1	2	3	4
Erreichte Punkte					

Teil 1
Multiple Choice

Hinweise zum Multiple-Choice-Teil

- Der Multiple-Choice-Teil wird **separat benotet** und bestimmt **1/3** der Gesamtnote.
- Pro Antwortmöglichkeit gilt: **richtig angekreuzt** oder **richtig nicht angekreuzt** wird positiv gewertet; **falsch angekreuzt** oder **falsch nicht angekreuzt** wird negativ gewertet.
- Pro Frage können **ein bis alle** Antwortmöglichkeiten korrekt sein. Kreuzen Sie alle zutreffenden Antworten an.

(a) Kreuzen Sie alle Kategorien nicht-funktionaler Anforderungen an.

- ☑ Produktanforderungen
- ☑ Unternehmensanforderungen
- ☐ Technische Anforderungen
- ☑ Externe Anforderungen

(b) Kreuzen Sie alle korrekt strukturierten User Stories an.

- ☐ Als Mensa-Manager möchte ich eine coole App, um die Mensa zu verwalten.
- ☑ Als Mensa-Besucherin möchte ich die Speisekarte der nächsten Woche einsehen können, um zu entscheiden, ob ich in der Mensa essen möchte.
- ☐ Als Mensa-Mitarbeiter möchte ich die Speisekarte der nächsten Woche festlegen können.
- ☐ Die Mensa-App soll die Speisekarte der nächsten Woche anzeigen können.

(c) Kreuzen Sie alle Bestandteile einer Szenariobeschreibung in der Anforderungs-Spezifikation an.

- ☑ Initiale Annahme
- ☐ Problemstellung
- ☐ Stakeholder
- ☑ Was falsch laufen kann
- ☑ Systemzustand bei Beendigung
- ☐ Verwandte Szenarien

(d) Kreuzen Sie die drei Schritte in der Anforderungserhebung an.

- ☑ Anforderungserhebung, Spezifikation, Validierung
- ☐ Anforderungsspezifikation, Verifikation, Validierung
- ☐ Anforderungsspezifikation, Entwurf, Implementierung
- ☐ Interview, Ethnographische Studie, Prototyping

(e) Welche Architektursicht hat folgende Architekturbeschreibung?
Die Software läuft auf einem Server mit Intel Core i5 und 8 GB RAM

- ☑ Physische Sicht
- ☐ Logische Sicht
- ☐ Prozess-Sicht
- ☐ Entwicklungs-Sicht

(f) Welche der folgenden Persönlichkeitsmerkmale gehören zu den Big Five?

- ☑ Conscientiousness (Gewissenhaftigkeit)
- ☑ Extraversion (Extraversion)
- ☑ Agreeableness (Verträglichkeit)
- ☑ Need for Stability (Verlangen nach Stabilität)
- ☐ Self-Efficacy (Selbstwirksamkeit)

(g) Welche Maßnahmen fördern psychologische Sicherheit in einem Entwicklungsteam?

- ☑ Fehler werden als Lernchancen betrachtet und nicht bestraft
- ☐ Strikte Hierarchie und klare Machtverhältnisse
- ☑ Team-Mitglieder können Bedenken äußern ohne Angst vor Konsequenzen
- ☐ Der oder die Teamleiter:in gibt Ratschläge und Anweisungen

(h) Welche der Aussagen treffen auf die Micro-Service Architektur zu?

- ☐ Micro-Services verringern die Komplexität eines Systems
- ☑ Micro-Services kommunizieren über wohldefinierte Schnittstellen.
- ☑ Micro-Services können unabhängig voneinander deployed werden.
- ☐ Micro-Services erzeugen keinen Overhead im Vergleich zu monolithischen Architekturen.
- ☑ Micro-Services isolieren Fehler. Ein Absturz eines Services bedeutet nicht den Ausfall des gesamten Systems.

(i) Welche Vorteile bietet das Model-View-Controller (MVC) Muster?

- ☐ Das Muster verbessert die Performanz durch weniger Komponenten.
- ☑ Daten können unabhängig von ihrer Repräsentation geändert werden.
- ☑ Es können unterschiedliche Darstellungen derselben Daten gleichzeitig existieren.
- ☐ Das Muster reduziert die Anzahl der benötigten Klassen im System.

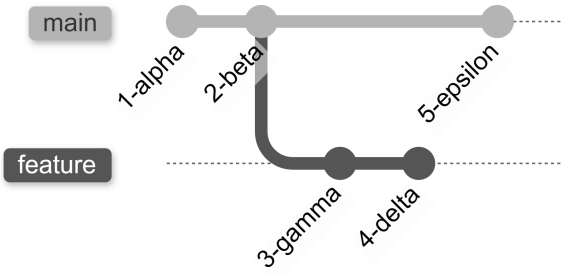
(j) Welche der folgenden Design Patterns gehören zu den Creational Patterns?

- ☑ Factory Method
- ☐ Observer
- ☑ Singleton
- ☐ Decorator
- ☑ Builder

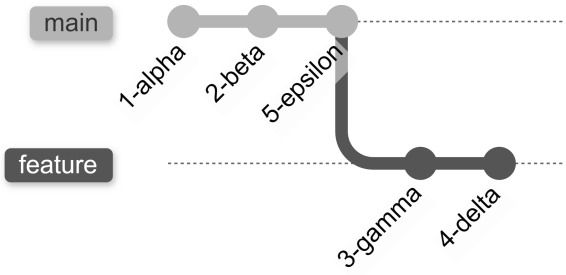
(k) Welche der folgenden Aussagen zu `merge` in Git sind korrekt?

- ☑ Bei einem merge werden die Historien von zwei Branches zusammengeführt.
- ☐ Ein merge erzeugt immer einen neuen Commit.
- ☐ Ein merge verändert den aktuellen Branch nicht.
- ☑ Bei einem merge können Merge-Konflikte auftreten.
- ☑ Ein merge kann rückgängig gemacht werden.

(l) Betrachten Sie die folgenden zwei Abbildungen:



(a)



(b)

Was ist beim Übergang von (a) zu (b) zu beobachten?

- ☐ Der Main-Branch wurde auf den Feature-Branch rebased
- ☑ Der Feature-Branch wurde auf den Main-Branch rebased
- ☐ Der Feature-Branch wurde mit dem Main-Branch gemerged
- ☐ Der Main-Branch wurde mit dem Feature-Branch gemerged

(m) Mit welchen Befehlen synchronisieren Sie Ihren lokalen Branch mit einem einem remote Repository?

- ☐ `git merge`
- ☐ `git commit`
- ☑ `git pull`
- ☐ `git add`
- ☑ `git push`

Bewertung	Punkte
4	35
3	41
2	47
1	53

Teil 2
Freie Aufgaben

Hinweise zu den freien Aufgaben

- Die freien Aufgaben werden **separat benotet** und bestimmen **2/3** der Gesamtnote.

Aufgabe 1: Software-Entwurf (10 Punkte)

(a) Welches Pattern stellt dieser Code dar? (1 Punkt)

```
public class Config {
  private static Config instance;

  private Config() {}

  public static Config getInstance() {
    if (instance == null) {
      instance = new Config();
    }
    return instance;
  }
}
```

Singleton Pattern

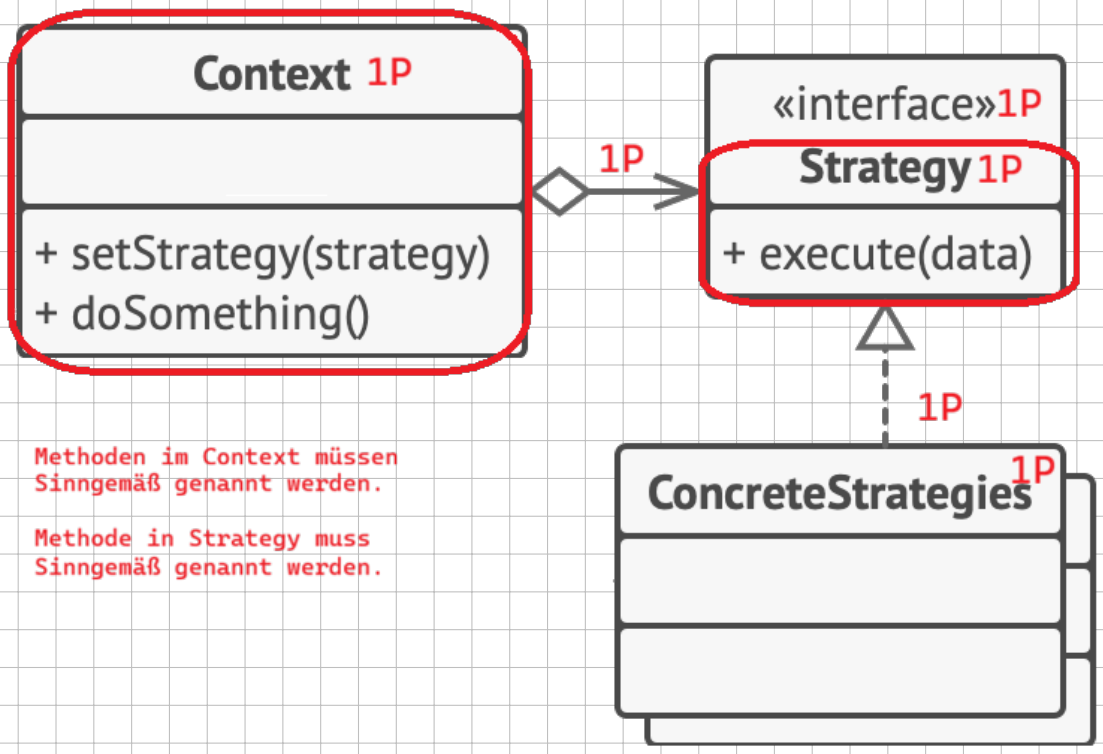
(b) Welches Design Pattern verwenden Sie für das folgende Szenario? Begründen Sie Ihre Entscheidung kurz. (3 Punkte)

> **Szenario**
>
> Sie möchten einfache Terme wie z. B. 3 + 5 oder 7 * 2 modellieren. Dabei kann ein Term aus Literalen und binären Operationen wie Addition und Multiplikation bestehen. Terme sind rekursiv definiert, das heißt, dass die Operanden einer binären Operation selbst wieder Terme sein können.

Lösung: Composite Pattern (1P): die Terme stellen eine Baumstruktur (1P) dar. Terme können transparent benhandelt werden (1P), egal wie tief Sie verschachtelt sind oder ob die Top-Level komponente ein Literal oder eine Operation ist.

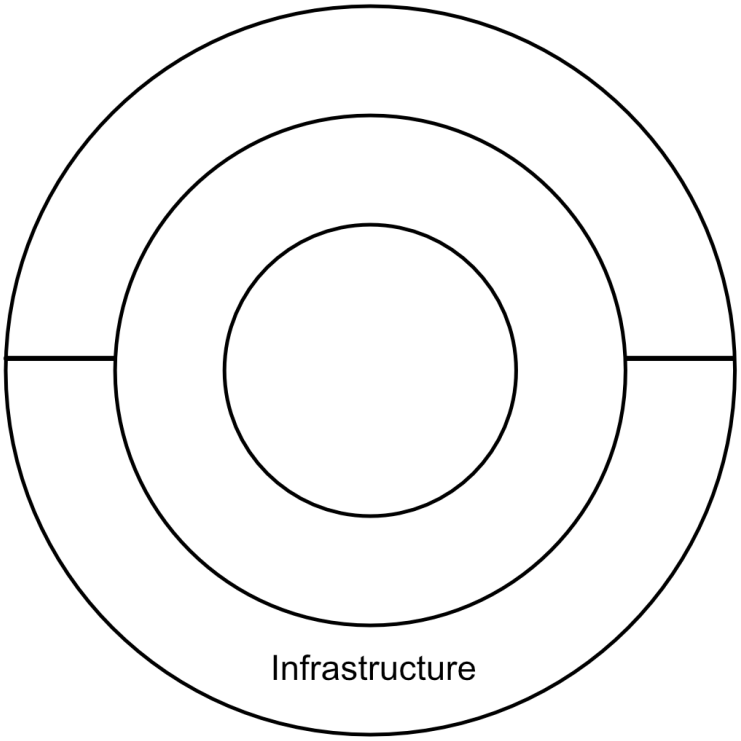
Aufgabe 1, Fortsetzung: Software-Entwurf (10 Punkte)

(c) Stellen Sie das Strategy Pattern als UML Klassendiagramm dar (inkl. wichtige Methoden aber ohne deren Inhalt, sowie ohne Multiplizitäten und Rollennamen). (6 Punkte)



Aufgabe 2: Softwarearchitektur (5 Punkte)

(a) Füllen Sie die fehlenden Bezeichnungen in der folgenden Abbildung der Onion-Architektur aus. (3 Punkte)



(b) Welches Arichtekturmuster wüden Sie in folgendem Szenario anwenden? Begründen Sie Ihre Wahl kurz. (2 Punkte)

> **Szenario**
>
> Es soll ein einfacher CSV-Verarbeitungstool entwickelt werden. Sie lesen eine CSV-Datei Zeilenweise ein, dann teilen Sie die Zeile an den Kommas ihre einzelnen Felder auf. Anschließend filtern Sie die Felder nach Ganzzahlen (integer).

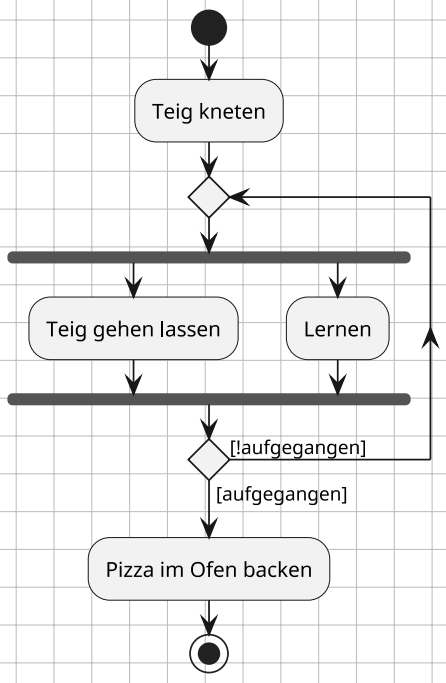
Lösung:

Pipes and Filters (1P).
Verarbeitung erfolgt in mehreren Schritten bei denen Daten von einem Schritt zum nächsten weitergereicht werden (1P).

Aufgabe 3: Modellierung mit UML (10 Punkte)

(a) Beschreiben Sie folgenden Sachverhalt aus der echten Welt als UML Aktivitätsdiagramm. Stellen Sie sicher, dass das Aktivitätsdiagramm alle darstellbaren Aspekte des Sachverhalts modelliert. (6 Punkte)

> **Szenario**
>
> Sie wollen eine Pizza backen. Zuerst kneten Sie den Teig. Sie lassen den Teig gehen und lernen währenddessen für Ihre Prüfung. Nachdem Sie eine weile gelernt haben, sehen sie nach dem Teig. Wenn der Teig aufgegangen ist, dann backen Sie eine Pizza im Ofen. Wenn der Teig nicht aufgegangen ist, dann lassen Sie ihn erneut gehen und lernen währenddessen. Diesen Prozess wiederholen Sie, bis der Teig aufgegangen ist.



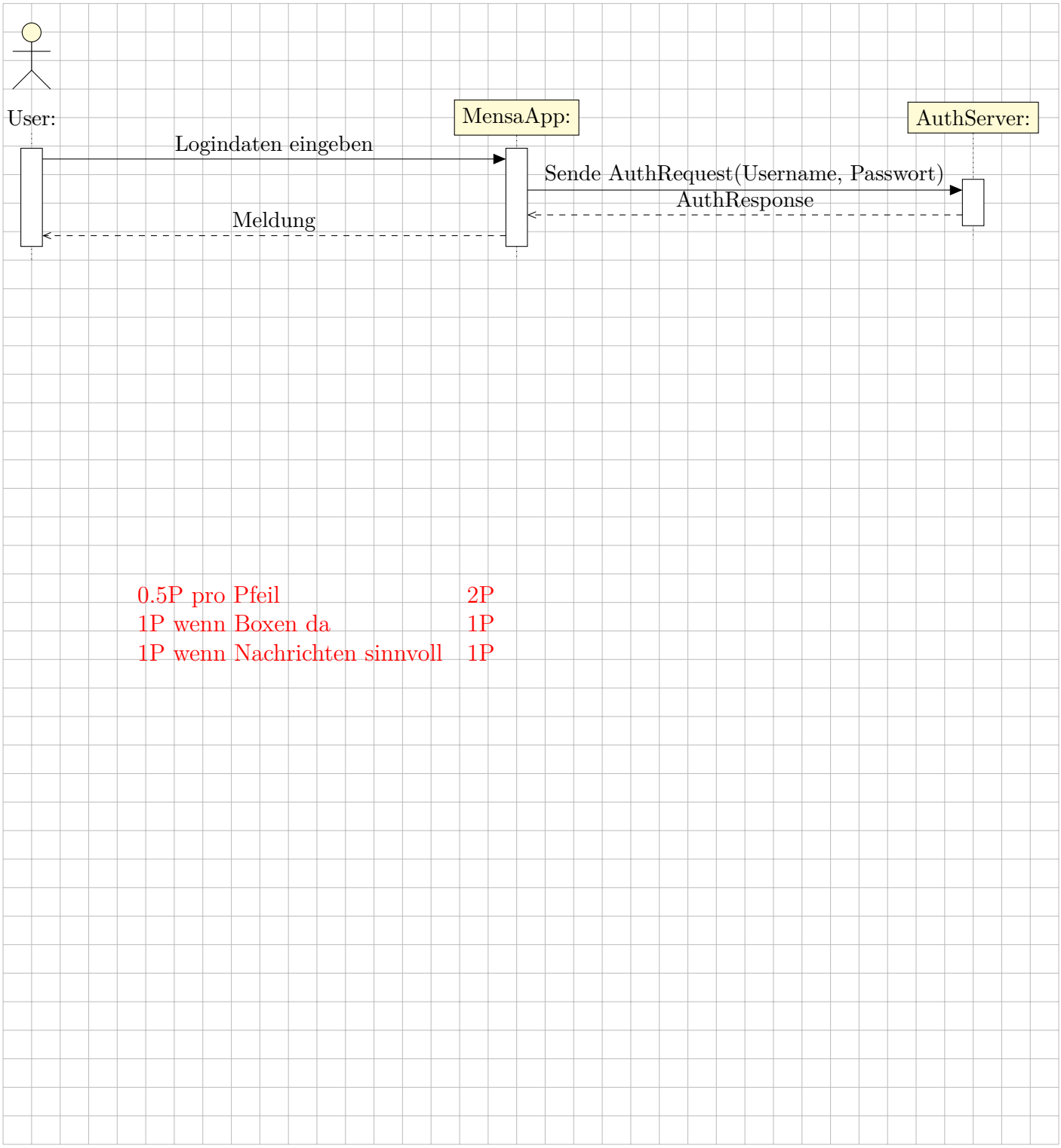
0.5P pro Aktivität	2P	Wenn zusätzliche Sinnvolle Knoten z.B. “Teig überprüfen” eingefügt werden, so gibt es keine extra Punkte darauf
1P Fork/Join korrekt eingesetzt	1P	
1P Fallunterscheidung (inkl. Bedingung)	1P	
1P Schleife korrekt eingesetzt	1P	
1P Start	1P	

Aufgabe 3, Fortsetzung: Modellierung mit UML (10 Punkte)

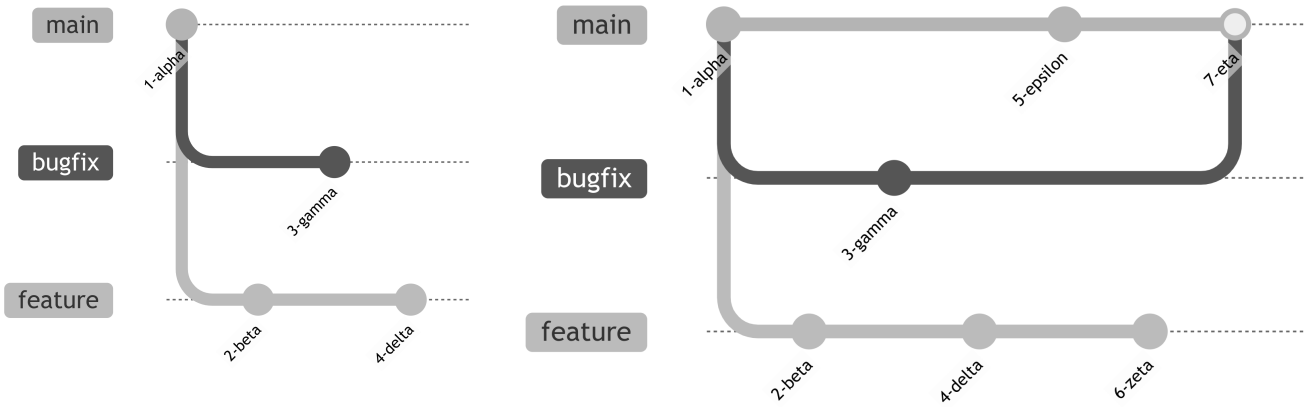
(b) Modellieren Sie den folgenden Sachverhalt in einem Sequenzdiagramm. Ergänzen Sie dazu die untenstehende Vorlage. (4 Punkte)

Szenario

Eine Nutzer:in möchte sich in der Mensa App einloggen. Dazu gibt sie ihren Benutzernamen und ihr Passwort ein. Die App sendet die Login-Daten an den Authentifizierungsserver. Dieser antwortet mit einer AuthResponse, welche angibt, ob die Authentifizierung erfolgreich war. Anschließend zeigt die App eine Erfolgs- bzw. Fehlermeldung an.



Aufgabe 4: Versionskontrolle (5 Punkte)



(a) (b)

Abbildung 1: Git-Repository mit zwei Zuständen.

a) Geben Sie die Git-Befehle an um von Zustand a zu Zustand b zu gelangen. Sie wissen nicht auf welchem branch Sie sich zu Beginn befinden. Gehen Sie davon aus, dass die Zahl am Anfang der Commits die chronologische Reihenfolge der Commits angibt. (5 Punkte)

Lösung: Punkte nur, wenn der Befehl in richtig in der Reihenfolge eingeordnet ist.

(1) `git switch main`, 1P

(2) `git commit`, 0.5P

(3) `git switch feature`, 1P

(4) `git commit`, 0.5P

(5) `git switch main`, 1P

(6) `git merge bugfix`, 1P